

Population protocols: Linear combinations. Semi-linear sets. Presburger's arithmetic.

September 2010

Abstract

These are lecture notes which are in preliminary form, and in particular sometimes mixing French and English.

1 How to Compute $\sum_{i=1}^k a_i x_i < c$

1.1 Formal protocol

Write input alphabet $\Sigma = \{\sigma_1, \dots, \sigma_k\}$. Let x_i be the number of letters σ_i in the initial population. The $a_1, a_2, \dots, a_k, c$ are integers. We want to build protocol that outputs eventually 1 if $\sum_{i=1}^k a_i x_i < c$ and 0 otherwise.

We do this as follows. Let $s = \max(|c| + 1, |a_1|, \dots, |a_k|)$.

We consider the population protocol $(Q, \Sigma, \delta, I, O)$ with

- State space

$$Q = \{0, 1\} \times \{0, 1\} \times \{u \in \mathbb{Z}, -s \leq u \leq s\}.$$

A state (l, o, c) encodes in l what we call the *leader bit*, in o what we call the *output bit*, and in c what we call the *count field*, or *value*.

- We take $I(\sigma_i) = (1, 0, a_i)$
- We take $O(l, b, c) = b$ for all l and c .
- The program σ is given by transition rules:

$$(l, \cdot, u), (l', \cdot, u') \rightarrow (1, b(u, u'), q(u, u'))(0, b(u, u'), r(u, u'))$$

if at least one of l or l' is 1.

(If both l and l' are 0, then do nothing).

where:

- for any integers, u, u' with $-s \leq u, u' \leq s$,

$$q(u, u') := \max(-s, \min(s, u + u'))$$

and

$$r(u, u') := u + u' - q(u, u')$$

- $b(u, u') = 1$ if $q(u, u') < c$ and 0 otherwise

Observe that $q(u, u')$ and $r(u, u')$ lie in $[-s, \dots, s]$, and that we always have

$$u + u' = q(u, u') + r(u, u').$$

1.2 Correctness of the protocol

1.2.1 Fact 1: The protocol converges to a single leader:

Let $\Lambda(C)$ be the set of agents whose leader bit equal 1 in configuration C . In the initial configuration c_0 , we have $|\Lambda(c_0)| = n$. Any encounter reduces cardinal of Λ by 1. If there are at least two leaders, by fairness they eventually meet. After finitely many steps, $|\Lambda(C)| = 1$.

1.2.2 Fact 2: The count field of the single leader converges to $\max(-s, \min(s, \sum_i a_i x_i))$:

- Given some agent j of configuration C , note $u_j = u_j(C)$ its count field.
- The protocol is build so that sum of agent's values is invariant. More formally,

$$\sum_j u_j = \sum_i a_i x_i$$

is an invariant.

- We consider the function p defined over configuration defined as follows:

$$p(C) := \sum_{j \notin \Lambda(C)} |u_j(C)|.$$

We will write p instead of $p(C)$ when configuration C is clear.

- Call a configuration C stable if there is a unique leader l and either

1. $p = 0$
2. $u_l = s$ and $u_j \geq 0$ for all $j \neq l$
3. $u_l = -s$ and $u_j \leq 0$ for all $j \neq l$

(u_l is the count field of the leader)

- By a simple inspection, observe that in a stable configuration,

$$u_l = \max(-s, \min(s, \sum_i a_i x_i)).$$

- The protocol converges to a stable configuration: indeed, from any configuration with a unique leader that is not stable, there is a transition that reduces p and no transition increase p .

This follows from the following arguments:

- p can decrease.

By inspection:

- * suppose $u_l = s$ and there some $j \neq l$ with $u_j < 0$: encounter l and j sets count fields to $s+u_j$ and 0. This reduces p by $-u_j > 0$
- * suppose $u_l = -s$ and there some $j \neq l$ with $u_j > 0$: encounter l and j sets count fields of j to 0. This reduces p by $u_j > 0$
- * suppose $-s < u_l < s$ and there some $j \neq l$ with $u_j \neq 0$: encounter l and j :
 - if $u_j > 0$ then p drops by $\min(u_j, s - u_l) > 0$
 - if $u_j < 0$ then p drops by $\min(-u_j, s + u_l) > 0$
- No other transition can increase p . By inspection.
 - * encounter between two non-leader: p is unchanged
 - * a leader l with $u_l = s$ and a $u_j \geq 0$: p is unchanged
 - * a leader l with $u_l = -s$ and a $u_j \leq 0$: p is unchanged.

1.2.3 Fact 3: Convergence of the output to the correct value

- In stable configuration, if $\sum_i a_i x_i < c$ then the leader count field is either $\sum_i a_i x_i$ or $-s < c$. In either cases, $b(u_l + u_j)$ gives the correct output, and any encounter between a leader a non-leader sets the output bit of both agents to 1.

No other transition sets the output bit of any agent to 0, and by fairness the leader eventually encounters all agents.

- Alternatively, if $\sum_i a_i x_i > c$ then the leader count field is either $\sum_i a_i x_i$ or $s > c$. In either cases, $b(u_l + u_j)$ gives the correct output, and any encounter between a leader a non-leader sets the bit of both agents to 0.

No other transition sets the output bit of any agent to 1, and by fairness the leader eventually encounters all agents.

1.3 How to Compute $\sum_{i=1}^k a_i x_i \equiv c[m]$

This is done in a very similar way (and this is easier).

Formally: Take now $s = \max(|c| + 1, |a_1|, \dots, |a_k|, m)$.

Take rule

$$(l, \cdot, u)(l', \cdot, u') \rightarrow (1, b, (u + u')[m])(0, b, 0)$$

if at least one of l and l' is 1,

where b is

- 1 if $u + u' \equiv c[m]$,
- 0 otherwise.

and $(u + u')[m]$ denotes $u + u'$ modulo m .

And do a similar reasoning.

2 Where are we?

2.1 What we get

From above constructions, we get the proof of the indirect sense of Theorem 1.

That is to say,

Proposition 1 *If a set is on the following list:*

- $\sum_{i=1}^k c_i x_i < c$, where c, c_i 's are integer constants
- $\sum_{i=1}^k c_i x_i \equiv c[m]$ where c, m and the c_i 's are integer constants
- Boolean combinations of the above predicates

then it is (stably) computable by a (2-way) population protocol.

2.2 Proving reverse direction

Proving reverse direction of Theorem 1 ((stably) computable by a (2-way) population protocol implies in the list) is not easy.

Actually, instead of proving directly reverse direction, we will reformulate Theorem 1, and see actually the result of Theorem 1 in a very more general settings.

We will first see that Theorem 1 is equivalent to say that (stably) computable predicates by a (2-way) population protocol are exactly semi-linear sets.

We will then see that Theorem 1 is equivalent to say that (stably) computable predicates by a (2-way) population protocol are exactly those that can be defined in Presburger's arithmetic.

3 Semi-linear sets

3.1 Working over $\mathbb{Z}^k$

We want to talk about (stably) computable predicates by population protocols.

Such predicates are predicates over $Pop(\Sigma)$ where Σ is the input alphabet.

Recall that a predicate ψ over $Pop(\Sigma)$ is a function from $Pop(\Sigma)$ to $\{0, 1\}$. Its support is the set of inputs that are mapped to 1, that is to say $\psi^{-1}(\{1\})$.

Recall that $Pop(\Sigma)$ can be seen as

$$Pop(\Sigma) := \mathbb{N}^\Sigma \setminus \{0\}.$$

As Σ can be considered to be $\Sigma = \{1, 2, \dots, k\}$ where k is the number of letters in the input alphabet Σ , it can also be seen as

$$Pop(\Sigma) := \mathbb{N}^k \setminus \{0\},$$

that is to say as a subset of $\mathbb{N}^k$, or $\mathbb{Z}^k$, where $\mathbb{N}$ is the set of (natural) integers, and $\mathbb{Z}$ the set of integers.

As a consequence, we are talking about classes of predicates over $\mathbb{Z}^k$, for some k . Considering that predicates are in one to one correspondance with their supports, in other words, we are talking about subsets of $\mathbb{Z}^k$, for some k .

3.2 Linear sets

The main class we will consider is the class of *semi-linear* predicates, denoted by *SLIN*.

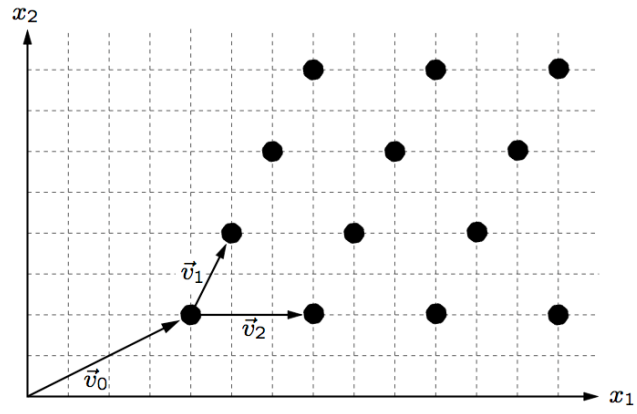
A set S of vectors $\vec{x} = (x_1, x_2, \dots, x_k) \in \mathbb{Z}^k$ is *linear* if it is of the form

$$S = \{\vec{v}_0 + c_1 \vec{v}_1 + c_2 \vec{v}_2 + \dots + c_m \vec{v}_m : c_1, \dots, c_m \in \mathbb{N}\}$$

where the $\vec{v}_i$ are vectors.

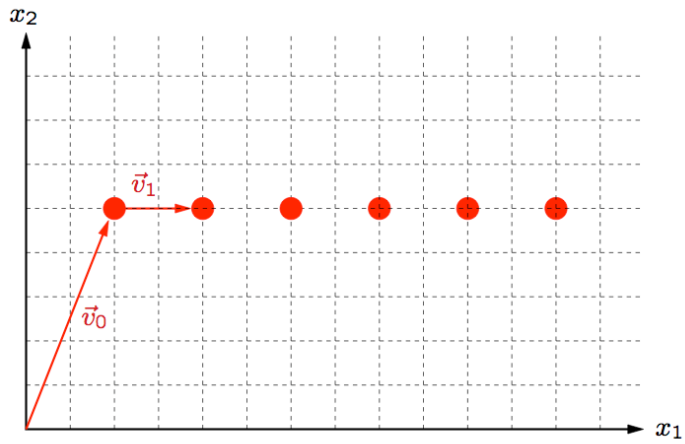
Vector $\vec{v}_0$ is called the *basis* of the linear set, and the $\vec{v}_i$'s are called the *periods*.

Here is for example the graphical representation of a linear set in $\mathbb{Z}^2$, of basis $(4, 2)$ and of periods $(3, 0)$ and $(1, 2)$:



$$S = \{\vec{v}_0 + c_1\vec{v}_1 + c_2\vec{v}_2 : c_1, c_2 \in \mathbb{N}\}$$

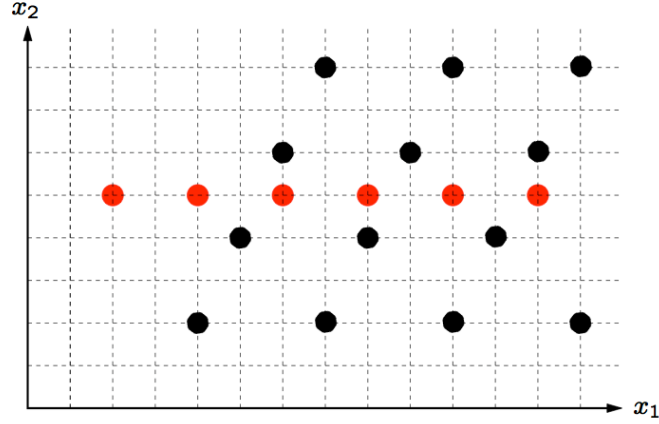
Here is another example:



$$T = \{\vec{v}_0 + c_1\vec{v}_1 : c_1 \in \mathbb{N}\}$$

A *semi-linear* set is a finite union of linear sets.

Here is an example (made of the union of two previous examples):



$$S \cup T$$

A predicate is said to be *semi-linear* if its support is.

3.3 Another formulation of Theorem 1

It turns out that Theorem 1 can also be re-formulated as follows.

Theorem 1 *A predicate is (stably) computable by a (two-way) population protocol if and only if it is semi-linear.*

3.4 Why?

The deep reason is that the following is known to be true:

Proposition 2 *A set $S \subset \mathbb{Z}^k$ is semi-linear if and only if it corresponds to (the support of) a predicate on the following list:*

1. $\sum_{i=1}^k c_i x_i < c$, where c , c_i 's are integer constants
2. $\sum_{i=1}^k c_i x_i \equiv c[m]$ where c , m and the c_i 's are integer constants
3. or boolean combinations of above predicates.

3.5 Proving Proposition 2

This is possible to prove Proposition 2 from scratch.

A proof could go as follows:

3.5.1 Indirect sense

Prove that

- semi-linear sets are closed by union (trivial, by definition).
- semi-linear sets are closed by complement (do a picture to be convinced).
- semi-linear sets are closed by intersection (tedious but not so hard algebra).
- hence by boolean operations.
- solutions of equations of Type 1. and 2. are semi-linear.

3.5.2 Direct sense

Since a semi-linear set is the finite union of linear sets, this is sufficient to prove statement (*): Any linear sets corresponds to (the support of) a predicate made of conjunctions of predicates of the type 1. and 2.

This turns out to be a special case of a more general result. We first talk about this latter result.

In Section 6, we will then come back and show

- that indirect sense (above) is actually clearly a special case of this more general result.
- how goes the proof of the direct sense, by treating completely an example.

4 Yet another formulation of Theorem 1

Indeed, this is possible to reformulate Theorem 1 in yet another framework, that turns out to be very more general in scope: (stably) computable predicates by a (2-way) population protocol are exactly those that can be defined in Presburger's arithmetic.

This is basically using the known (but rather surprising) fact that semi-linear sets corresponds precisely to (support of) the predicates that are definable in Presburger's arithmetic.

4.1 Presburger's arithmetic

Informally, a predicate is definable in Presburger's arithmetic if and only if it can be expressed in (first-order) logic using the symbols $+$, 0 , 1 , $\vee$, $\wedge$, $\neg$, $\forall$, $\exists$, $=$, $<$, $(,)$ and variables, that is to say with all the symbols of classical (first-order) logic but multiplication.

Remark: $\vee$ stands for the symbol for logical disjunction, $\wedge$ for logical conjunction, $\neg$ for logical negation.

4.1.1 Peano's arithmetic

This is possible to skip this subsection, as there is no need to talk about multiplication in order to forget it. However, from an historical perspective, this discussion can help. This is inspired from the presentation of Peano's arithmetic in Wikipedia.

Peano introduced in the 19th century a set of axioms (that everybody assumed to be true) for the natural numbers. First four statements are about equality sign $=$, that is stated to reflexive, transitive, symmetric, and such that natural numbers are closed under equality. The next four axioms are first-order statements about natural numbers expressing the fundamental properties of the successor operation, that is to say about $s(x) := x + 1$. The ninth, final axiom is a second order statement of the principle of mathematical induction over the natural numbers. Then Peano's arithmetic is obtained by adding an operation of addition and multiplication. Addition satisfy (and is uniquely defined by): $a + 0 = a$ and $a + s(b) = s(a + b)$. Multiplication satisfy (and is uniquely defined by) $a \times 0 = 0$, $a \times s(b) = a + a \times b$.

When the Peano axioms were first proposed, Bertrand Russell and others agreed that these axioms implicitly defined what we mean by a "natural number". Henri Poincaré was more cautious, saying they only defined natural numbers if they were consistent; if there is a proof that starts from just these axioms and derives a contradiction such as $0 = 1$, then the axioms are inconsistent, and don't define anything. In 1900, David Hilbert posed the problem of proving their consistency (using only finistic methods) as the second of his twenty-three problems. In 1931, Kurt Gödel proved his second incompleteness theorem, which shows that such a consistency proof cannot be formalized within Peano arithmetic itself.

In other words, Peano's arithmetic can be considered as the the first-order theory of the natural numbers with addition and multiplication.

The vast majority of contemporary mathematicians believe that Peano's axioms are consistent.

Peano's arithmetic is not complete: this is not true that for each statement in Peano's arithmetic, either it is possible to deduce it from the axioms or it is possible to deduce its negation.

Peano's arithmetic is known to be undecidable: There exists no algorithm which decides whether any given statement in Peano's arithmetic is true or false.

4.1.2 Presburger's arithmetic

Presburger's arithmetic can be considered as the first-order theory of the natural numbers with addition. Hence, the signature of Presburger arithmetic contains only the addition operation and equality, omitting the multiplication operation entirely.

Presburger arithmetic is much weaker than Peano arithmetic, which includes both addition and multiplication operations. Unlike Peano arithmetic, Presburger arithmetic is a decidable theory: This means it is possible to effectively

determine, for any sentence in the language of Presburger arithmetic, whether it is true or false.

Presburger's arithmetic is known to be consistent (there is no statement in Presburger arithmetic which can be deduced from the axioms such that its negation can also be deduced) and complete (for each statement in Presburger arithmetic, either it is possible to deduce it from the axioms or it is possible to deduce its negation).

4.2 Presburger's arithmetic and semi-linear sets

We want to consider formulas from Presburger's arithmetic (with free variables) and we want to associate to each such formula the set of valuations of (free) variables that makes it true.

For example, the following formula ψ

$$(\exists z : (z \geq 0) \wedge (x = z + z + 1) \wedge (y \geq z))$$

$$\vee (\exists z : (z \geq 0) \wedge (x = z + z) \wedge (y = 1 + 1))$$

is a formula from Presburger's arithmetic with free variables x and y .

Remark: Observe that we replaced multiplications by some constant c by c times addition.

Remark: to be very formal, as we stated (informally) that Presburger's arithmetic involves formulas with only $+, 0, 1, \vee, \wedge, \neg, \forall, \exists, =, <, (,)$ and variables, we should say that $x \geq y$ is an abbreviation for $\neg(y < x)$.

We want to associate to this formula the set of integers x and y that makes it true.

This turns out to be exactly the semi-linear set

$$S = \{(1, 0) + c_1(0, 1) + c_2(2, 1) | c_1, c_2 \in \mathbb{N}\} \cup \{(0, 2) + c_3(2, 0) | c_3 \in \mathbb{N}\}.$$

The following is known:

Proposition 3 (Ginsburg & Spanier) *A predicate is semi-linear if and only if it corresponds to a formula of Presburger's arithmetic.*

4.3 What we get

We get the promised reformulation of Theorem 1.

Theorem 2 *A predicate is (stably) computable by a (two-way) population protocol if and only if it corresponds to a formula of Presburger's arithmetic.*

that is to say, informally, iff it can be expressed in first-order logic using the symbols $+, 0, 1, \vee, \wedge, \neg, \forall, \exists, =, <, (,)$ and variables (but no multiplication).

4.3.1 Applications

This yields immediately computability for example of:

- Majority: if we write $\Sigma = \{B, R\}$, then telling if there are more R than B corresponds to Presburger's formula $x_1 < x_2$, hence is computable.
- Divisible by 3: telling whether there is a number of 1 in the initial population divisible by 3 corresponds to Presburger's formula $\exists y : y + y + y = x_1$, hence is computable.
- At least 40% 1's: telling whether there is a number of 1 in the initial population which is greater than 40% the number of 0 and 1 corresponds to Presburger's formula $\neg(x_1 + x_1 < x_0 + x_0 + x_0)$, hence is computable.

An example of a predicate that is not definable by a Presburger's formula (and hence is not computable) is for example $x \times x \geq y$: this involves a multiplication, that can not be changed in additions (unlike multiplications by constants that can be changed in finitely many additions).

5 Why?

We do not intend to prove Proposition 3. This would lead to discussions out of the scope of the ambition of this course (and actually, would oblige to be really more formal about logic).

For a self-content proof, not so hard to read, refer to pages 152 to 158 of [4].

We can however give some intuitions.

5.1 The easy sense

The easy direction of Proposition 3 is to state that a semi-linear set corresponds to a Presburger's arithmetic formula.

Actually, this is sufficient to prove that a linear set corresponds to a Presburger's arithmetic formula, as a semi-linear set is a union of linear sets, and hence corresponds to the $\vee$ of the formulas defining these linear sets.

But this is easy, as a linear set over $\mathbb{Z}^k$ is of the form

$$S = \{\vec{v}_0 + c_1 \vec{v}_1 + c_2 \vec{v}_2 + \dots + c_m \vec{v}_m : c_1, \dots, c_m \in \mathbb{N}\}$$

and hence corresponds to formula

$$\begin{aligned} \exists c_1 \exists c_2 \dots \exists c_m (x_1 = v_{01} + c_1 v_{11} + \dots + c_m v_{m1} \wedge \dots \wedge \\ x_k = v_{0k} + c_1 v_{1k} + \dots + c_m v_{mk}) \wedge \\ c_1 \geq 0 \wedge c_2 \geq 0 \dots \wedge c_m \geq 0 \end{aligned}$$

where $\vec{x} = (x_1, x_2, \dots, x_k)$, and $\vec{v}_i = (v_{i,1}, v_{i,2}, \dots, v_{i,k})$, for $i = 0, 1, \dots, m$, and

- $v_{0,j}$ is an abbreviation for $1 + 1 + \dots + 1$ where in this last expression 1 appears $v_{0,j}$ times.
- $c_i v_{i,j}$ is an abbreviation for $c_i + c_i + \dots + c_i$ where in this last expression c_i appears $v_{i,j}$ times.

For example, the linear set of basis $(4, 2)$ and periods $(3, 0)$ and $(1, 2)$ corresponds to

$$\exists n_1 \exists n_2 (n_1 \geq 0 \wedge n_2 \geq 0 \wedge x_1 = 1 + 1 + 1 + 1 + n_1 + n_1 + n_1 + n_2 \wedge x_2 = 1 + 1 + n_2 + n_2)$$

Let's rewrite such a formula more succinctly as

$$\exists n_1 \exists n_2 \in \mathbb{N} \begin{cases} x_1 &= 4 + 3n_1 + 1n_2 \\ x_2 &= 2 + 0n_1 + 2n_2 \end{cases}$$

5.2 The hard sense

The hard direction of Proposition 3 is to prove that any set defined by a Presburger's arithmetic formula is semi-linear. This is done basically by proving that Presburger's arithmetic admits quantifier elimination: any formula from Presburger's arithmetic can be written without any quantifier if one adds some symbols $\equiv_m$ to test equality modulo m , for all integer m .

In other words, any formula with existential (or universal) quantifiers can be transformed into a formula without any quantifier using such new symbols.

Once again, for a self-content proof, not so hard to read, refer to pages 152 to 158 of [4].

Once again, we do not intend to prove it. However, let us give some tricks used in the proof.

Here is a first trick:

5.2.1 Trick 1

How to eliminate $\exists n_1 \in \mathbb{N}$ in

$$\exists n_1 \in \mathbb{N} \begin{cases} x_1 &= n_1 v_1 + n_2 w_1 + u_1 \\ x_2 &= n_1 v_2 + n_2 w_2 + u_2 \end{cases}$$

We can rewrite this problem into

$$\exists n_1 \in \mathbb{N} \begin{cases} n_1 v_1 &= x_1 - n_2 w_1 - u_1 \\ n_1 v_2 &= x_2 - n_2 w_2 - u_2 \end{cases}$$

An by suitable multiplication into

$$\exists n_1 \in \mathbb{N} \begin{cases} n_1 \nu &= x'_1 - n_2 w'_1 - u'_1 \\ n_1 \nu &= x'_2 - n_2 w'_2 - u'_2 \end{cases},$$

with $\nu > 0$.

This rewrites to

$$\begin{cases} x'_2 - n_2 w'_2 - u'_2 & = & x'_1 - n_2 w'_1 - u'_1 \\ x'_2 - n_2 w'_2 - u'_2 & \equiv & 0 \ [\nu] \\ x'_1 - n_2 w'_1 - u'_1 & \geq & 0 \end{cases}$$

5.2.2 Trick 2

Let u, v, w, x be integers. Let p be the great common multiple of w and x . There exists m and n such that $1 = mp/w + np/x$.

Then a trick is that

$$\begin{cases} y \equiv u[w] \\ y \equiv v[x] \end{cases}$$

is equivalent to

$$\begin{cases} u \equiv v[\gcd(w, x)] \\ y \equiv m(p/w)u + n(p/x)v[p]. \end{cases}$$

this gives a way to eliminate a conjunction of congruences to only one with a given variable.

6 Back to Proposition 2

6.1 Direct sense

We promised to come back to statement (*): Any linear sets corresponds to (the support of) a predicate made of conjunctions of predicates of the type 1. and 2.

This follows mainly from Trick 1 mentioned above.

Instead of doing a formal proof in the general case, let's show how it works on an example.

Take previous example of linear set. This was the linear set of basis $(4, 2)$ and periods $(3, 0)$ and $(1, 2)$.

We stated it can be written as follows:

$$\exists n_1 \exists n_2 \in \mathbb{N} \begin{cases} x_1 & = & 4 + 3n_1 + 1n_2 \\ x_2 & = & 2 + 0n_1 + 2n_2 \end{cases}$$

Let's use Trick 1 to eliminates the quantification over n_2 .

Indeed, let's rewrite previous formula to

$$\exists n_1 \exists n_2 \in \mathbb{N} \begin{cases} 1n_2 & = & x_1 - 4 - 3n_1 \\ 2n_2 & = & x_2 - 2 \end{cases}$$

Multiply line 1 by 2

$$\exists n_1 \exists n_2 \in \mathbb{N} \begin{cases} 2n_2 & = & 2x_1 - 8 - 6n_1 \\ 2n_2 & = & x_2 - 2 \end{cases}$$

Rewrites to

$$\exists n_1 \in \mathbb{N} \left\{ \begin{array}{lcl} 2x_1 - 8 - 6n_1 & = & x_2 - 2 \\ x_2 - 2 & \equiv & 0[2] \\ x_2 - 2 & \geq & 0 \end{array} \right.$$

If one prefers

$$\exists n_1 \in \mathbb{N} \left\{ \begin{array}{lcl} 2x_1 - 8 - 6n_1 & = & x_2 - 2 \\ x_2 & \equiv & 0[2] \\ x_2 - 2 & \geq & 0 \end{array} \right.$$

Ok, we are done.

Let's use Trick 1 to eliminates the quantification over n_1 now.

Rewrites all this to

$$\exists n_1 \in \mathbb{N} \left\{ \begin{array}{lcl} 6n_1 & = & 2x_1 - 6 - x_2 \\ x_2 & \equiv & 0[2] \\ x_2 - 2 & \geq & 0 \end{array} \right.$$

That is:

$$\left\{ \begin{array}{lcl} 2x_1 - 6 - x_2 & \geq & 0 \\ 2x_1 - 6 - x_2 & \equiv & 0[6] \\ x_2 & \equiv & 0[2] \\ x_2 & \geq & 2 \end{array} \right.$$

If one prefers:

$$\left\{ \begin{array}{lcl} 2x_1 - 6 - x_2 & \geq & 0 \\ 2x_1 - x_2 & \equiv & 0[6] \\ x_2 & \equiv & 0[2] \\ x_2 & \geq & 2 \end{array} \right.$$

This is indeed a formula obtained as a conjunctions of formulas as stated by Proposition 2.

We leave to our reader to be convinced that this works for any linear set.

6.2 Indirect sense

If we admit Proposition 3 then the indirect sense of Proposition 2 is easy.

The proof goes as follows:

- solutions of equations of Type 1. and 2. corresponds to Presburger's arithmetic formulas
- sets defined by Presburger's arithmetic formulas are clearly closed by boolean operations (conjunction corresponds to $\wedge$, negation to $\neg$, and disjunction to $\vee$)

Hence a set in the list corresponds to a set defined by a Presburger's arithmetic formulas, hence is semi-linear.

References

- [1] D. Angluin, J. Aspnes, D. Eisenstat, and E. Ruppert. The computational power of population protocols. *Distributed Computing*, 20(4):279–304, 2007.
- [2] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. In *Twenty-Third ACM Symposium on Principles of Distributed Computing*, pages 290–299. ACM Press, July 2004.
- [3] James Aspnes and Eric Ruppert. An introduction to population protocols. In *Bulletin of the EATCS*, volume 93, pages 106–125, 2007.
- [4] M. Kracht and G.K. Pullum. The mathematics of language. *The Mathematical Intelligencer*, 28(2):74–78, 2006.